UNIVERSIDADE ESTADUAL DA PARAÍBA
CENTRO DE CIÊNCIA E TECNOLOGIA - CCT
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

GUILHERME SILVA HENRIQUES DE SOUZA
KAYLLANE ARAÚJO DOS SANTOS
LUIS GUSTAVO SOUZA LOURENÇO

DOCENTE: FÁBIO LUIZ LEITE JÚNIOR

Relatório submetido no curso de Ciência da Computação da Universidade Estadual da Paraíba, na disciplina de Técnicas de Análise de Algoritmos, correspondente ao período de 2025.2.

CAMPINA GRANDE
2026

SUMÁRIO

1. INTRODUÇÃO.....2
1.1. Contextualização do Problema.....2
1.2. Objetivos..... 2
1.2.1. Geral.....2
1.2.2. Específico.....2
2. METODOLOGIA E IMPLEMENTAÇÃO..... 3
2.1. Estrutura do Sistema Desenvolvido..... 3
2.2. Formato de Dados de Entrada e Saída.....4
2.3. Métricas de Avaliação..... 4
3. ALGORITMOS IMPLEMENTADOS.....6
3.1. Backtracking.....6
3.2. Branch and Bound.....7
3.3. Programação Dinâmica.....8
3.4. Estratégia Gulosa.....9
4. EXPERIMENTOS E RESULTADOS..... 10
5. ANÁLISE E DISCUSSÃO.....13
6. CONCLUSÃO..... 14
7. REFERÊNCIAS..... 15

1. INTRODUÇÃO

1.1. Contextualização do Problema

O presente trabalho tem como objetivo apresentar o estudo das N-RAINHAS, em que as rainhas são posicionadas em um tabuleiro NxN de tal forma que duas rainhas não podem se cruzar diagonalmente, nem em linhas e colunas. Este problema é um clássico da inteligência artificial, frequentemente utilizado para estudar busca, backtracking e otimização.

1.2. Objetivos

1.2.1. Geral

Analisar e compreender o problema das N-RAINHAS, explorando suas restrições e métodos de resolução por meio de técnicas computacionais utilizadas na Ciência da Computação e na Inteligência Artificial.

1.2.2. Específico

- Implementar soluções para o problema das N-RAINHAS utilizando diferentes estratégias de projeto de algoritmos:
  - Backtracking
  - Branch and Bound
- Realizar uma comparação formal entre as técnicas implementadas, considerando diferentes critérios de avaliação.
- Analisar a complexidade assintótica teórica dos algoritmos utilizados.
- Medir e comparar tempo de execução empírico das soluções implementadas.
- Comparar a qualidade das soluções obtidas, verificando se são ótimas ou aproximadas.
- Avaliar a escalabilidade dos algoritmos, observando seu comportamento para diferentes tamanhos de tabuleiro.
- Analisar as podas realizadas no espaço de busca, identificando como cada técnica reduz o número de soluções exploradas.
- Apresentar uma justificativa técnica explicando de que forma cada estratégia contribui para otimizar a busca no espaço de possíveis soluções.

2. METODOLOGIA E IMPLEMENTAÇÃO

2.1. Estrutura do Sistema Desenvolvido

O sistema desenvolvido foi implementado na linguagem Java, com o objetivo de comparar diferentes técnicas de algoritmos aplicados ao problema das N-RAINHAS. O programa foi estruturado de forma modular, seguindo princípios de orientação a objetos. A execução é iniciada por uma classe principal chamada NQueens, responsável apenas pela interação com o usuário e pelo controle do fluxo geral da aplicação. A lógica de resolução do problema foi abstraída em uma classe base chamada NQueensSolver, que define a estrutura comum para os algoritmos, incluindo métricas de desempenho e métodos auxiliares para exibição dos resultados.

As estratégias de resolução foram implementadas em classes específicas que estendem a classe base:

- BacktrackingSolver, que utiliza a abordagem clássica de backtracking;
- BranchAndBoundSolver, que aplica a técnica de branch and bound com otimizações no controle de conflitos
- DynamicProgrammingSolver, implementa a técnica de memorização para evitar o reprocessamento de estados de sub tabuleiros já analisados.
- GreedySolver, aplica uma escolha local imediata para o posicionamento das rainhas, priorizando a primeira posição segura encontrada sem realizar retrocessos.

Dessa forma, o sistema permite:

- Receber o valor de N informado pelo usuário por meio da classe principal;
- Selecionar dinamicamente o algoritmo a ser executado;
- Executar cada estratégia de forma independente;
- Medir o tempo de execução de cada algoritmo;
- Contabilizar o número de estados explorados e backtracks realizados;

- Exibir os resultados e o tabuleiro solução de forma organizada.

Essa organização torna o código mais modular, reutilizável e fácil de expandir, permitindo a adição de novos algoritmos sem impactar a estrutura principal da aplicação.

2.2. Formato de Dados de Entrada e Saída

O programa recebe como entrada um valor inteiro N, que representa o tamanho do tabuleiro e a quantidade de rainhas a serem posicionadas no problema das N-Rainhas. Esse valor é informado pelo usuário durante a execução do programa. Caso N seja maior que zero, o algoritmo é executado; caso N seja igual a zero, o programa é encerrado. Após cada execução, o usuário pode escolher testar outro valor de N.

Como saída, o programa executa quatro estratégias de resolução: Backtracking, Branch and Bound, Programação Dinâmica e Estratégia Gulosa. Para cada uma delas, são exibidas a solução encontrada, a representação do tabuleiro, o tempo de execução, o número de nós explorados e a quantidade de backtracks, que indica quantas vezes o algoritmo precisou voltar para tentar outras possibilidades. Essas informações permitem comparar o desempenho das quatro técnicas.

2.3. Métricas de Avaliação

Para realizar a comparação entre as diferentes técnicas de projeto de algoritmos, foram utilizadas algumas métricas de avaliação durante a execução do programa. A primeira métrica analisada foi o tempo de execução, que foi medido utilizando o método System.nanoTime(). Esse método permite registrar com alta precisão o instante de início e fim da execução do algoritmo, possibilitando calcular o tempo necessário para encontrar a solução. Posteriormente, esse valor é convertido para milissegundos (ms) para facilitar a análise e comparação entre os resultados.

Outra métrica utilizada foi a quantidade de estados explorados, representada no código pela variável nodesExplored. Essa variável contabiliza quantas posições do espaço de busca foram analisadas pelos algoritmos durante o processo de resolução do problema. A partir dessa métrica, é possível avaliar a eficiência da busca, a quantidade de tentativas realizadas e o impacto das podas no espaço de busca, especialmente no caso do algoritmo Branch and Bound, que utiliza técnicas para reduzir o número de estados que precisam ser explorados. Além disso, também foi observada a quantidade de backtracks, que indica quantas vezes o algoritmo precisou retornar a um estado anterior para tentar uma nova possibilidade de posicionamento das rainhas.

3. ALGORITMOS IMPLEMENTADOS

3.1. Backtracking

O backtracking é uma técnica baseada em busca recursiva no espaço de soluções. Esse método constrói soluções de forma incremental, testando possíveis configurações e retrocedendo (backtrack) sempre que identifica que uma determinada escolha levará a uma solução inválida.

No contexto das N-RAINHAS, o algoritmo posiciona uma rainha em cada linha do tabuleiro, verificando se a posição escolhida não entra em conflito com a posição de rainhas já posicionadas anteriormente. Caso seja detectado algum conflito, o algoritmo descarta essa posição e tenta outra possibilidade.

Essa abordagem garante que todas as combinações válidas sejam exploradas de forma sistemática, permitindo encontrar uma solução correta para o problema. No entanto, o espaço de busca cresce rapidamente com o aumento de N, o que pode resultar em um alto custo computacional em termos de tempo de execução para valores maiores de entrada.

No sistema desenvolvido, o backtracking foi implementado utilizando recursão, onde cada chamada da função tenta posicionar uma rainha em uma nova linha do tabuleiro e verifica se a posição é segura antes de prosseguir para a próxima etapa da busca.

Em relação à corretude, o algoritmo garante que nenhuma solução inválida seja aceita, pois antes de posicionar cada rainha é realizada uma verificação de segurança que impede conflitos de coluna ou diagonal com rainhas já posicionadas. Dessa forma, quando o algoritmo alcança a última linha do tabuleiro, tem-se uma configuração válida para o problema.

Quanto ao desempenho, o algoritmo pode apresentar crescimento exponencial no número de estados explorados, uma vez que diversas combinações precisam ser testadas até que uma solução válida seja encontrada. Assim, à medida que o valor de N aumenta, cresce também a

quantidade de tentativas e retrocessos realizados pelo algoritmo, impactando diretamente no tempo de execução.

3.2. Branch and Bound

O Branch and Bound é uma técnica utilizada para resolver problemas de busca e otimização, baseada na exploração sistemática do espaço de soluções. A ideia principal consiste em dividir o problema em subproblemas menores (*branch*) e eliminar antecipadamente aqueles que não podem levar a uma solução válida (*bound*), reduzindo assim o número de possibilidades analisadas.

No problema das N-Rainhas, essa técnica funciona de forma semelhante ao backtracking, porém com um mecanismo mais eficiente de verificação de conflitos. Em vez de verificar repetidamente todas as rainhas já posicionadas, o algoritmo mantém estruturas auxiliares que indicam se determinadas colunas ou diagonais já estão ocupadas por outras rainhas.

No sistema desenvolvido, foram utilizados três vetores booleanos para realizar esse controle:

- cols: indica se uma coluna do tabuleiro já está ocupada por uma rainha.
- diag1: representa as diagonais principais do tabuleiro.
- diag2: representa as diagonais secundárias do tabuleiro.

Com essas estruturas, o algoritmo consegue verificar rapidamente se uma posição é válida antes de posicionar uma nova rainha. Caso uma coluna ou diagonal já esteja ocupada, aquela ramificação da busca é imediatamente descartada, caracterizando a poda no espaço de busca.

O algoritmo então prossegue posicionando a rainha apenas nas posições consideradas seguras. Caso não exista posição válida em determinada linha, o algoritmo retorna para a linha anterior e tenta uma nova possibilidade.

Em relação à corretude, o algoritmo garante que nenhuma configuração inválida seja aceita, pois antes de posicionar cada rainha é verificado se a

coluna e as diagonais correspondentes estão livres. Assim, quando o algoritmo consegue posicionar todas as rainhas no tabuleiro, tem-se uma solução válida para o problema.

Quanto ao desempenho, o uso das estruturas auxiliares permite reduzir o custo das verificações de conflito, tornando o processo de busca mais eficiente. Além disso, a poda antecipada de configurações inválidas diminui a quantidade de estados explorados, o que tende a reduzir o tempo de execução quando comparado a abordagens que realizam verificações repetitivas. Dessa forma, o Branch and Bound pode apresentar melhor desempenho na exploração do espaço de soluções, principalmente para valores maiores de N.

3.3. Programação Dinâmica

A programação dinâmica é uma técnica utilizada para otimizar problemas que apresentam sobreposição de subproblemas, evitando o recálculo de soluções já obtidas anteriormente. Essa abordagem baseia-se no armazenamento de resultados intermediários, permitindo reutilizar soluções parciais e reduzir o custo computacional.

No contexto do problema das N-Rainhas, a aplicação da programação dinâmica não é direta como em problemas clássicos, como Fibonacci ou mochila, pois o estado do problema depende da configuração completa do tabuleiro até o momento.

No sistema desenvolvido, a programação dinâmica foi implementada por meio de uma abordagem de memoização, na qual os estados já explorados são armazenados em uma estrutura de dados (tabela). Cada estado é definido pela combinação da linha atual e das estruturas que indicam as colunas e diagonais ocupadas. Caso o algoritmo encontre novamente um estado já resolvido anteriormente, o resultado armazenado é reutilizado, evitando recomputações desnecessárias.

O funcionamento do algoritmo segue uma lógica semelhante ao backtracking, posicionando uma rainha por linha e verificando se a posição é válida.

Entretanto, antes de explorar uma nova configuração, o algoritmo verifica se aquele estado já foi previamente analisado. Caso tenha sido, o resultado é recuperado diretamente da memória.

Em relação à corretude, o algoritmo garante que apenas configurações válidas sejam consideradas, uma vez que todas as verificações de conflito continuam sendo realizadas antes do posicionamento de cada rainha. Assim, quando todas as linhas são preenchidas, obtém-se uma solução válida para o problema.

3.4. Estratégia Gulosa

A estratégia gulosa (greedy) é uma técnica de projeto de algoritmos baseada na escolha da melhor opção local em cada etapa, com o objetivo de alcançar uma solução global. Diferentemente de abordagens como backtracking, essa técnica não realiza exploração completa do espaço de busca nem retrocessos, o que a torna geralmente mais rápida.

No problema das N-Rainhas, a abordagem gulosa consiste em posicionar cada rainha em uma linha do tabuleiro escolhendo, de forma direta, uma coluna que não gere conflito imediato com as rainhas já posicionadas. O algoritmo percorre o tabuleiro linha por linha e seleciona a primeira posição considerada válida, sem revisitar decisões anteriores.

No sistema desenvolvido, a estratégia gulosa foi implementada de forma sequencial, tentando posicionar uma rainha em cada linha sem realizar backtracking. Caso uma posição válida seja encontrada, o algoritmo prossegue para a próxima linha; caso contrário, a execução pode falhar ou depender de uma escolha inicial adequada para encontrar solução.

Em relação à corretude, a estratégia gulosa não garante encontrar solução para todas as entradas do problema, pois decisões locais podem impedir a construção de uma solução global válida. No entanto, nos experimentos realizados, foi possível obter soluções corretas para os valores testados, devido à ordem específica de escolha das posições.

Quanto ao desempenho, a principal vantagem dessa abordagem está na ausência de retrocessos, o que resulta em tempo de execução reduzido e número de backtracks igual a zero. Contudo, essa eficiência vem acompanhada da limitação de não garantir solução em todos os casos, sendo uma abordagem mais apropriada para obtenção rápida de soluções aproximadas ou quando combinada com outras técnicas.

4. EXPERIMENTOS E RESULTADOS

Para avaliar o comportamento dos algoritmos Backtracking, Branch and Bound, Programação Dinâmica e Estratégia Gulosa (Greedy) na resolução do problema das N-Rainhas, foram realizados experimentos utilizando diferentes valores de entrada para N, representando o tamanho do tabuleiro. O objetivo desses testes foi analisar como cada técnica se comporta conforme o tamanho do problema aumenta.

Durante os experimentos, foram coletadas principalmente duas métricas: nós explorados e número de backtracks. A métrica de nós explorados representa quantos estados da árvore de busca foram analisados pelo algoritmo até encontrar uma solução válida. Já o número de backtracks indica quantas vezes o algoritmo precisou desfazer uma escolha e retornar para um estado anterior da árvore de busca.

Os testes foram executados utilizando os valores N = 4, N = 8, N = 16 e N = 24, permitindo observar o crescimento do espaço de busca conforme o tamanho do tabuleiro aumenta.

Entrada para N = 4

Backtracking	Branch and Bound
Solução encontrada: 2 4 1 3 Tempo: 0,1037 ms Nós explorados: 26 Backtracks: 4	Solução encontrada: 2 4 1 3 Tempo: 0,1802 ms Nós explorados: 26 Backtracks: 22

Programação Dinâmica	Estratégia Gulosa (Greedy)
Solução encontrada: 2 4 1 3 Tempo: 16,5865 ms Nós explorados: 26 Backtracks: 4	Solução encontrada: 2 4 1 3 Tempo: 0,0923 ms Nós explorados: 26 Backtracks: 0

Entrada para N = 8

Backtracking	Branch and Bound
Solução encontrada: 1 5 8 6 3 7 2 4 Tempo: 0,5949 ms Nós explorados: 876 Backtracks: 105	Solução encontrada: 1 5 8 6 3 7 2 4 Tempo: 0,2183 ms Nós explorados: 876 Backtracks: 868

Programação Dinâmica	Estratégia Gulosa (Greedy)
Solução encontrada: 1 5 8 6 3 7 2 4 Tempo: 1,0729 ms Nós explorados: 852 Backtracks: 105	Solução encontrada: 1 5 8 6 3 7 2 4 Tempo: 0,3860 ms Nós explorados: 876 Backtracks: 0

Entrada para N = 16

Backtracking	Branch and Bound
Solução encontrada: 1 3 5 2 13 9 14 12 15 6 16 7 4 11 8 10 Tempo: 8,4725 ms Nós explorados: 160712 Backtracks: 10036	Solução encontrada: 1 3 5 2 13 9 14 12 15 6 16 7 4 11 8 10 Tempo: 3,8935 ms Nós explorados: 160712 Backtracks: 160696

Programação Dinâmica	Estratégia Gulosa (Greedy)
Solução encontrada: 1 3 5 2 13 9 14 12 15 6 16 7 4 11 8 10 Tempo: 29,4053 ms Nós explorados: 156104 Backtracks: 9816	Solução encontrada: 1 3 5 2 13 9 14 12 15 6 16 7 4 11 8 10 Tempo: 12,5910 ms Nós explorados: 160712 Backtracks: 0

Entrada para N = 24

Backtracking	Branch and Bound
Solução encontrada: 1 3 5 2 4 9 11 14 18 22 19 23 20 24 10 21 6 8 12 16 13 7 17 15 Tempo: 162,3061 ms Nós explorados: 9878316 Backtracks: 411584	Solução encontrada: 1 3 5 2 4 9 11 14 18 22 19 23 20 24 10 21 6 8 12 16 13 7 17 15 Tempo: 97,4262 ms Nós explorados: 9878316 Backtracks: 9878292

Programação Dinâmica	Estratégia Gulosa (Greedy)
Solução encontrada: 1 3 5 2 4 9 11 14 18 22 19 23 20 24 10 21 6 8 12 16 13 7 17 15 Tempo: 365,3909 ms Nós explorados: 9646644 Backtracks: 402438	Solução encontrada: 1 3 5 2 4 9 11 14 18 22 19 23 20 24 10 21 6 8 12 16 13 7 17 15 Tempo: 243,8157 ms Nós explorados: 9878316 Backtracks: 0

5. ANÁLISE E DISCUSSÃO

A partir dos experimentos realizados com diferentes valores de N, foi possível observar o comportamento dos algoritmos Backtracking, Branch and Bound, Programação Dinâmica e Estratégia Gulosa na resolução do problema das N-Rainhas.

Para valores menores de N, como N = 4 e N = 8, todos os algoritmos apresentaram desempenho semelhante, encontrando soluções rapidamente devido ao espaço de busca reduzido. Nesses casos, as diferenças entre as abordagens são pouco perceptíveis.

Entretanto, à medida que o valor de N aumenta, torna-se evidente o impacto das estratégias utilizadas. Os algoritmos Backtracking e Programação Dinâmica apresentam crescimento significativo no número de estados explorados e de backtracks, refletindo o caráter combinatório do problema. Embora a programação dinâmica reduza a repetição de alguns estados, o ganho de desempenho é limitado, pois o espaço de estados continua muito grande.

O algoritmo Branch and Bound, por sua vez, apresenta melhor eficiência nas verificações de conflito, devido ao uso de estruturas auxiliares que permitem identificar rapidamente posições inválidas. Apesar de explorar uma quantidade semelhante de nós nesta implementação, o custo das verificações é reduzido, o que impacta positivamente o tempo de execução.

Já a estratégia gulosa se destaca por não realizar backtracking, o que resulta em execução mais rápida e menor complexidade operacional. No entanto, essa abordagem não garante encontrar solução em todos os casos, pois suas decisões são baseadas apenas em escolhas locais, sem considerar o impacto global.

De forma geral, os resultados demonstram que o problema das N-Rainhas possui alta complexidade combinatória, e que diferentes técnicas de projeto de algoritmos influenciam diretamente na forma de exploração do espaço de soluções e no desempenho da execução.

6. CONCLUSÃO

Neste trabalho foram estudadas e implementadas quatro técnicas de projeto de algoritmos aplicadas ao problema das N-Rainhas: Backtracking, Branch and Bound, Programação Dinâmica e Estratégia Gulosa.

O Backtracking demonstrou ser uma abordagem robusta e completa, capaz de encontrar soluções válidas por meio da exploração sistemática do espaço de busca, embora com custo elevado para valores maiores de N. O Branch and Bound apresentou melhorias na eficiência das verificações, reduzindo o custo computacional ao eliminar rapidamente configurações inválidas.

A Programação Dinâmica, embora aplicada com o objetivo de evitar recomputações, mostrou-se menos eficiente para este problema específico, devido à complexidade do estado necessário para representar as soluções parciais e ao grande número de possibilidades envolvidas.

Por outro lado, a Estratégia Gulosa apresentou o melhor desempenho em termos de tempo de execução e ausência de retrocessos, porém sem garantia de encontrar solução em todos os casos, uma vez que suas decisões são baseadas apenas em escolhas locais.

Dessa forma, o estudo permitiu compreender que a escolha da técnica de projeto de algoritmos deve considerar não apenas o desempenho, mas também a garantia de corretude e a natureza do problema. No caso das N-Rainhas, abordagens baseadas em busca completa, como Backtracking e Branch and Bound, mostram-se mais adequadas para garantir soluções corretas, enquanto técnicas como Greedy podem ser utilizadas como alternativas rápidas em cenários específicos.

7. REFERÊNCIAS

DILLIHANGRAE. *N-Queen Problem (Backtracking Algorithm).* Medium, 16 ago. 2024. Disponível em: https://medium.com/@dillihangrae/n-queen-problem-backtracking-algorithm-55ef786c3116. Acesso em: 16 mar. 2026.

IACONO, Andrea. *Backtracking Explained.* Medium. Disponível em: https://medium.com/@andreaiacono/backtracking-explained-7450d6ef9e1a. Acesso em: 16 mar. 2026.

ANDRETTA, Marcelo. *Técnicas de Backtracking.* Instituto de Ciências Matemáticas e de Computação – Universidade de São Paulo (USP). Disponível em: https://sites.icmc.usp.br/andretta/ensino/aulas/sme0510-2-19/aula10.pdf. Acesso em: 16 mar. 2026.

GEEKSFORGEEKS. *Difference between Backtracking and Branch and Bound Technique.* Disponível em: https://www.geeksforgeeks.org/dsa/difference-between-backtracking-and-branch-n-bound-technique/. Acesso em: 16 mar. 2026.

TOSCANI, Laira Vieira; VELOSO, Paulo Augusto S.. Programação Dinâmica e Método Guloso. *In*: SIMPÓSIO BRASILEIRO DE ENGENHARIA DE SOFTWARE (SBES), 2. , 1988, Canela/RS. Anais [...]. Porto Alegre: Sociedade Brasileira de Computação, 1988 . p. 106-112. ISSN 2833-0633. DOI: https://doi.org/10.5753/sbes.1988.24249. Acesso em: 01 abril. 2026